DOCUMENTACIÓN TÉCNICA

Roguelike co-op

Guía para entender el código y hacer tus propios cambios: estructura, cómo funciona por dentro, recetas paso a paso y las reglas que no hay que romper.

Monorepo TypeScript · Babylon.js 9 · Colyseus 0.18 · Vite · Vitest
Estado del código: rama `main`, octubre 2026 (protocolo 14)

Contenido

1. Antes de empezar — instalar, correr, probar, el panel F2
2. La arquitectura en una página — quién manda, qué viaja por la red
3. Mapa del repositorio — qué hay en cada carpeta y archivo
4. El ciclo del servidor — un tick de `RunRoom`, paso a paso
5. El ciclo del cliente — un frame de `main.ts`, predicción e interpolación
6. Red y protocolo — el estado sincronizado, los mensajes, la versión
7. Determinismo y números aleatorios
8. Los sistemas del servidor — combate, habilidades, enemigos, oleadas, orbes, cartas
9. Dónde viven los datos del juego — las tablas que vas a tocar más
10. El cliente por dentro — vistas, animaciones, efectos, HUD
11. Recetas paso a paso — cómo agregar cartas, habilidades, enemigos, armas, equipo…
12. Tests
13. Reglas y trampas comunes — la lista de chequeo antes de cada cambio
14. Flujo de trabajo y documentos
15. Glosario

1. Antes de empezar

1.1 Requisitos e instalación

Necesitas **Node 24 o más nuevo**. El proyecto es un *monorepo* de npm (workspaces): tres paquetes que se instalan juntos con un solo comando desde la raíz.

```
cd roguelike-coop
npm install
```

1.2 Jugar en desarrollo

```
npm run dev
```

Levanta dos procesos a la vez:

- **Servidor** (Colyseus) en `ws://localhost:2567`, con `tsx watch`: se reinicia solo cuando guardas un archivo del servidor o de `shared`.
- **Cliente** (Vite) en `http://localhost:5173`: recarga la página sola cuando guardas un archivo del cliente.

Abre siempre `http://localhost:5173`. El puerto 2567 solo sirve el cliente compilado en producción.

1.3 Parámetros de URL útiles (solo en desarrollo)

Parámetro	Qué hace	Ejemplo
`sandbox=1`	Crea una **sala de pruebas**: empieza abierta (sin sellos), oleadas apagadas y el panel F2 disponible.	`?sandbox=1`
`floorTimeStart`	Crea la sala con el reloj del piso adelantado (segundos). Útil para ver oleadas tardías.	`?floorTimeStart=300`
`lag`, `jitter`	Simula latencia (ms de ida y vuelta) y variación, para probar la predicción.	`?lag=150&jitter=30`

Se pueden combinar: `?sandbox=1&floorTimeStart=400`.

1.4 El panel de pruebas (F2)

En una sala `?sandbox=1`, F2 abre un panel a la derecha. Cada botón manda un comando al servidor (mensaje `dev`); el servidor lo valida y lo aplica, así que lo que ves es lo mismo que pasaría jugando. Secciones:

- **Nivel**: subir 1 nivel (con elección de carta y pausa) o 5 niveles sin cartas.
- **Cartas**: darte cualquier carta o habilidad en cualquier rango; quitarlas todas.
- **Jugador**: invulnerable, sin enfriamiento, curar, y aplicarte estados (quemadura, ralentizar, aturdir).
- **Enemigos**: crear N enemigos de un tipo, tier, naturaleza y opcionalmente élite; matar a todos; "Enemigos quietos"; "Hechizo ya"; oleadas on/off.
- **Reloj del piso**: +1 / +5 minutos.

- **Arma**, **Cofres**, **Equipo**: equiparte cualquier arma/pieza/reliquia, soltar piezas, crear un cofre delante.

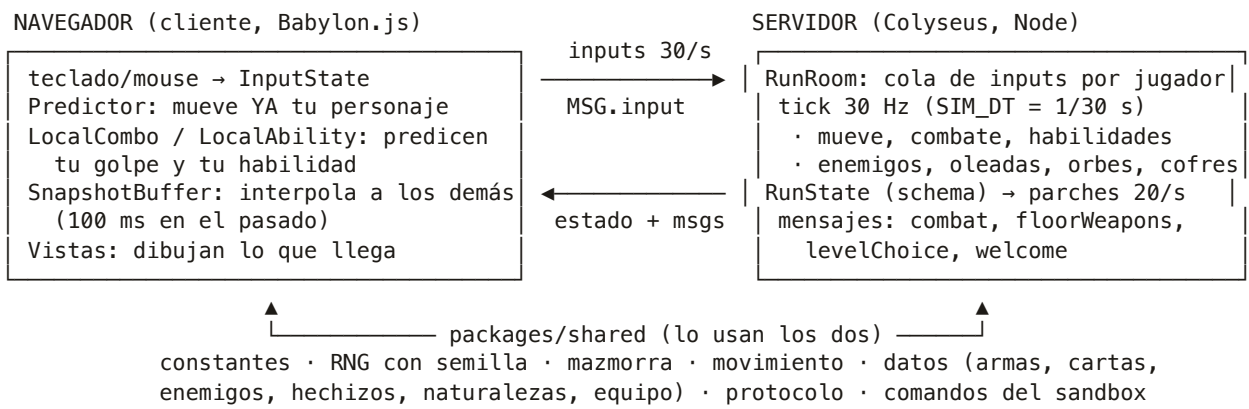
El panel está en `packages/client/src/ui/devPanel.ts`; los comandos se definen y validan en `packages/shared/src/sandbox.ts` y se aplican en `RunRoom.applyDev` (servidor).

1.5 Tests, tipos y compilación

```
npm test                          # los tests de los tres paquetes
npm test -w @roguelike/server     # solo un paquete
cd packages/server && npx vitest run test/EnemySystem.test.ts   # un archivo
cd packages/client && npx tsc --noEmit -p .                     # chequeo de tipos
npm run build -w @roguelike/client                               # compila el cliente
```

Ojo con el puerto 2568. Los tests de `RunRoom.test.ts` levantan un servidor de prueba en el puerto **2568**. Si tienes algo escuchando ahí, esos tests no arrancan.

2. La arquitectura en una página



2.1 Las cuatro ideas que explican todo

1. **El servidor es autoritativo.** Él decide posiciones, daño, muertes, botín y niveles. El cliente nunca "decide" nada que importe; solo envía *inputs* (qué teclas apretaste) y dibuja.
2. **El cliente predice lo tuyo.** Para que tu personaje responda sin esperar al servidor, el cliente simula tu movimiento con **el mismo código** que el servidor (`stepPlayer` en `shared`). Cuando llega el estado real, **reconcilia**: parte de la posición del servidor y vuelve a aplicar los inputs que el servidor todavía no confirmó. Lo mismo hace con el inicio de tu golpe y de tus habilidades.
3. **Lo demás se interpola.** Otros jugadores, enemigos, proyectiles, etc., se dibujan 100 ms en el pasado (`INTERP_DELAY_MS`), interpolando entre dos estados recibidos. Así se ven suaves aunque lleguen 20 parches por segundo.
4. **`shared` es la fuente única de verdad de reglas y datos.** Si algo debe dar igual en cliente y servidor (movimiento, tablas de armas, fórmulas), vive en `shared` y es **determinista**.

2.2 Números clave (`packages/shared/src/constants.ts`)

Constante	Valor	Para qué
`SIM_HZ` / `SIM_DT`	30 / 1/30 s	Frecuencia de la simulación (servidor y predicción).
`PATCH_RATE_MS`	50	Cada cuánto el servidor manda el estado (20/s).
`INTERP_DELAY_MS`	100	Cuánto en el pasado se dibuja lo interpolado.
`MAX_INPUTS_PER_TICK`	3	Inputs de un jugador procesados por tick como máximo.
`TILE_SIZE`	2 m	Tamaño de una casilla de la mazmorra.
`PLAYER_SPEED`, `SPRINT_SPEED`	5, 8 m/s	Velocidades del jugador.
`JUMP_VELOCITY`, `GRAVITY`	7 m/s, 20 m/s²	Salto (~1,1 m de alto).
`PICKUP_RANGE`	1,5 m	Distancia para recoger con E y abrir cofres.
`REWIND_MAX_TICKS`	10	Cuánto rebobina el servidor para compensar el lag al golpear.
`RESPAWN_S`	5 s	Tiempo hasta reaparecer.
`MAX_PLAYERS`	4	Jugadores por sala.

2.3 Compensación de lag al golpear

Cuando haces clic, ves a los enemigos donde estaban hace ~100 ms (interpolados). Para que el golpe sea justo, cada input lleva `viewTick` (el tick que estabas viendo) y el servidor, al resolver el impacto, mira las posiciones guardadas en `PositionHistory` de ese momento (acotado a `REWIND_MAX_TICKS`). El daño se aplica en el **momento del impacto** del golpe (una fracción de la animación), no en el clic.

3. Mapa del repositorio

```
roguelike-coop/
├── package.json            scripts de raíz (dev, test, build) y workspaces
├── README.md               cómo jugar, controles, varias pestañas, lag
├── CREDITS.md              licencias de modelos y animaciones
├── docs/
│   ├── specs/              un documento de diseño por fase (qué y por qué)
│   ├── plans/              el plan de implementación de cada fase (cómo, por tareas)
│   └── guia-tecnica/       esta guía (HTML + PDF)
├── scripts/assets/build-mixamo.mjs   arma los .glb de personajes, animaciones y armas
└── packages/
    ├── shared/   reglas y datos comunes (sin dependencias de motor ni de red)
    ├── server/   la sala de Colyseus y la simulación
    └── client/   el juego en el navegador (Babylon.js + DOM para la UI)
```

3.1 `packages/shared/src`

Archivo	Contenido
`constants.ts`	Constantes globales (ver 2.2).
`rng.ts`	`createRng(seed)`: generador determinista (mulberry32) con `next`, `int`, `pick`.
`dungeon.ts`	Generador de mazmorras por semilla: casillas (0 muro, 1 suelo), salas (`start` / `combat` / `boss`), pasillos. `isWalkableTile`, `tileCenter`, `roomSpawnPoints`.
`movement.ts`	`PlayerInput`, `KinematicState`, `stepPlayer` (mover + saltar + colisiones), `sanitizeInput`, `deadInput`, `moveCircle`.
`nav.ts`	`FlowField` (BFS hacia los jugadores, para que los enemigos esquiven muros) y `hasLineOfSight`.
`combat.ts`	Clases (`CLASS_DEFS`: vida por nivel), `ENEMY_DEFS` (vida, velocidad, daño, alcance, telegrafía…), `xpToNext`, `playerStats`.
`weapons.ts`	`MOVESETS` (animaciones y combo de cada estilo), `WEAPONS`, `TIERS` (oxidado → legendario), `strikeOf`, `strikeDamage`, armas de enemigos.
`cards.ts`	Cartas de stat (`STAT_CARDS`), habilidades (`ABILITIES`), `rollOffer`, `applyCard`, `playerMods`.
`waves.ts`	Curvas del director de oleadas, escalado de enemigos, tope de vivos, sorteo de tiers, hechiceros por minuto, probabilidad de élite.
`spells.ts`	Hechiceros (`SPELLS`: orbe, zona, rayo por tier), `casterTelegraphS`.
`natures.ts`	Naturalezas (fuego/hielo/rayo), `NATURE`, `ELITE`, `natureEffects`.
`gear.ts`	Equipo (ranuras, stats por tier, legendarias, reliquias), `rollGear`, `combinedMods`.
`chests.ts`, `start.ts`	Posición de cofres y botín; sellos de la sala inicial.
`protocol.ts`	`PROTOCOL_VERSION`, nombres de mensajes (`MSG`), `CombatEvent`, validación de nombres.
`sandbox.ts`	Comandos del panel F2 (`DevCommand`) y `parseDevCommand`.

3.2 `packages/server/src`

Archivo	Contenido
`index.ts`, `app.config.ts`	Arranque del servidor y registro de la sala `run`.
`rooms/RunRoom.ts`	**El corazón**: crea la partida, recibe inputs, corre el tick, reparte botín, aplica comandos del sandbox.
`rooms/RunState.ts`	El **estado sincronizado** (schema de Colyseus): jugadores, enemigos, proyectiles, orbes, peligros, cofres, zonas…
`rooms/syncState.ts`	Copia la simulación al schema cada tick (`syncPlayer`, `syncWorld`).
`rooms/PlayerRegistry.ts`	Registro de jugadores por nombre/token (reconexión), su progreso, equipo, objetos del suelo.
`rooms/createOptions.ts`, `sandbox.ts`	Opciones de creación (semilla, `sandbox`, `floorTimeStart`) y utilidades del sandbox.
`sim/CombatSystem.ts`	Vida, combo de ataques, daño recibido (con bloqueo, resistencias, estados), muertes, XP del grupo.
`sim/AbilitySystem.ts`	Habilidades activas: enfriamientos, cargas, impactos, Grito, ondas de corte, zonas del jugador.
`sim/EnemySystem.ts`	Enemigos: creación, IA, ataques, proyectiles, peligros (zonas y rayos), estados, élites, botín.
`sim/hazards.ts`	Geometría de zonas y rayos (`inZone`, `inBeam`, `beamLength`).
`sim/WaveDirector.ts`	Cuándo y dónde aparecen las oleadas (fuera de la vista de los jugadores).
`sim/OrbSystem.ts`	Orbes de XP: imán, recogida, fusión.
`sim/LevelUpSystem.ts`	Elección de cartas al subir de nivel (pausa del grupo, 20 s, ofertas sin pausa).
`sim/PositionHistory.ts`	Historial de posiciones de enemigos para la compensación de lag.

3.3 `packages/client/src`

Carpeta / archivo	Contenido
`main.ts`	Entrada: lobby, conexión, carga de modelos, `startSession` (crea todas las vistas y el bucle `frame`).
`net/connection.ts`	Conexión a Colyseus, **snapshots** tipados del estado, parseo defensivo de mensajes, envío de inputs y comandos.
`net/Predictor.ts`	Predicción y reconciliación de tu movimiento.
`net/LocalCombo.ts`, `LocalAbility.ts`	Predicción del inicio del golpe y de las habilidades (con enfriamientos y cargas).
`net/SnapshotBuffer.ts`, `fixedStep.ts`, `pauseClock.ts`, `tickClock.ts`	Interpolación, paso fijo de 30 Hz, reloj de juego que se congela en la pausa.
`input/`	`InputState` (estado de teclas, una pulsación = una acción) y `browserInput` (eventos del navegador, puntero).
`render/`	Escena, cámara, mazmorra, personajes, animaciones y todos los efectos (ver capítulo 10).
`ui/`	Interfaz en DOM: HUD, barra de habilidades, panel de cartas, brújula, minimapa, panel F2, lobby, iconos.
`index.html`	Estructura de la página y **todo el CSS** del HUD (variables `--gold`, `--ink`, fuentes).
`public/assets/`	Modelos .glb (personajes, animaciones, armas, mazmorra) e imágenes de la UI.
`dev/grip.html`	Página de desarrollo para ajustar cómo se agarran las armas.

4. El ciclo del servidor

`RunRoom` llama a `tick()` 30 veces por segundo (`setTimestep`). Este es el orden real, y el orden importa: el cliente predice en el mismo orden.

4.1 Un tick, paso a paso

1. **Elecciones de cartas** (`levelUp.step`). Si el grupo está en **pausa** eligiendo cartas, se corre `pausedTick` y el tick termina ahí: nada avanza, los inputs se descartan pero su `seq` se confirma.
2. **Inputs de cada jugador conectado** (hasta 3 por tick). Para cada input, en este orden:
    1. Si está muerto, el input se convierte con `deadInput` (quieto, sin acciones).
    2. `stepPlayer` mueve al jugador (con su multiplicador de velocidad: cartas, estados, bloqueo).
    3. `combat.onInputProcessed`: avanza el combo y resuelve impactos de golpes que vencen.
    4. `abilities.onInput`: habilidades (empezar, impactos, bloqueo) y un posible impulso (Tajo frontal).
    5. E: abrir cofre o recoger objeto (si no está golpeando ni en una habilidad).
    6. Ataque: empezar un golpe nuevo del combo.
3. `state.tick++`.
4. **Inicio de la partida**: si alguien salió de la sala inicial, `started = true`. Antes de empezar no corren reloj, salas ni oleadas.
5. **Reloj del piso** (`floorTime += SIM_DT`) y escalado de enemigos por tiempo y nivel.
6. **Salas y oleadas**: `enemySystem.activateRooms` (llenar una sala al entrar) y `director.step` (grupos que llegan con el tiempo).
7. **Enemigos**: `enemySystem.step` (IA, ataques, proyectiles, peligros, estados) → sus golpes se encolan en el combate.
8. **Habilidades por tick** (enfriamientos, Grito, ondas de corte, zonas).
9. **Daño por estados** (quemaduras) y **orbes**: los vivos atraen y recogen XP; si el grupo sube de nivel, la pausa empieza al final del tick.
10. **Combate por tick**: golpes enemigos que vencen, muertes, reapariciones, regeneración.
11. **Historial** de posiciones (para la compensación de lag).
12. **Sincronizar** todo al schema (`syncPlayer`, `syncWorld`). Colyseus manda los cambios cada 50 ms.

4.2 Cómo entra un input

El cliente manda `MSG.input` con un `seq` creciente. `enqueueInput` lo pasa por `sanitizeInput` (nunca confíes en el cliente), descarta repetidos o viejos y lo encola. El tick consume la cola. El `lastSeq` de cada jugador viaja en el estado para que el cliente sepa qué inputs ya se procesaron.

4.3 Jugadores: registro y reconexión

Un jugador se identifica por **nombre + token** (guardado en `localStorage`). `PlayerRegistry` conserva su `PlayerRecord` aunque se desconecte: posición, combate, progreso (cartas), equipo, reliquia, objetos del

suelo. Volver con el mismo nombre y token lo recupera; con el mismo nombre desde otra pestaña "toma el control" y la anterior se cierra.

4.4 La pausa de elección de cartas

Al subir de nivel, `LevelUpSystem` pausa al grupo: cada jugador elige 1 de 3 cartas (20 s, al azar si no elige). Mientras dura, el servidor no avanza nada y el cliente congela su reloj de juego (`PauseClock`): animaciones, partículas y efectos se detienen. **Todo lo nuevo que dependa del tiempo debe respetar esta pausa** (ver 13).

5. El ciclo del cliente

5.1 Arranque

1. `main.ts` muestra el lobby (`ui/lobby.ts`).
2. `connect()` (`net/connection.ts`) crea o une la sala y espera `welcome` (tu clave y si es sandbox).
3. Se cargan modelos (`loadAssets`) y animaciones (`AnimLibrary`).
4. `startSession` genera la **misma mazmorra** que el servidor a partir de la semilla (`generateDungeon(seed, floor)`) y crea todas las vistas, el HUD y el bucle.

5.2 Los manejadores de red

- `conn.onPlayers`: llega tu estado y el de los demás. El tuyo va al `Predictor.reconcile`; los demás a un `SnapshotBuffer` cada uno.
- `conn.onWorld`: enemigos, proyectiles, orbes, peligros, cofres, zonas → se "empujan" (`push`) a sus vistas.
- `conn.onCombat`: eventos sueltos (daño, golpe, nivel, habilidad, cadena del rayo) → números de daño, efectos.
- `conn.onFloorWeapons`: tus objetos del suelo (cada jugador ve los suyos).
- `conn.onLevelChoice`: abre el panel de cartas.

5.3 Un frame, paso a paso (`frame()` en `main.ts`)

1. Calcula el tiempo de juego (`clock.now`, quieto en la pausa) y el `dt`.
2. **Paso fijo a 30 Hz** (`FixedStep`): por cada paso, toma un input (`input.sample`), lo sanea, lo predice (`predictor.applyLocal`), avanza el combo y la habilidad locales y lo envía (`conn.sendInput`). *Exactamente lo mismo, en el mismo orden, que hará el servidor.*
3. Dibuja tu personaje en la posición predicha (interpolada entre los dos últimos pasos, más un suavizado de correcciones).
4. Dibuja a los demás y al mundo en `renderAt = now − 100 ms` (interpolados).
5. Actualiza HUD, brújula, minimapa, barra de habilidades.
6. Avisa la pausa a todas las vistas (`setPaused`) y renderiza la escena.

5.4 Predicción: la regla de oro

> **Todo lo que el cliente predice debe calcularse igual que en el servidor.** Si cambias cómo se mueve el jugador (velocidad, bloqueo, estados), cómo empieza un golpe o una habilidad (enfriamientos, cargas), tienes que cambiarlo en los dos lados, idealmente con una función de `shared` que usen ambos (como `stepPlayer`, `deadInput` o `blockMoveMult`). Si no coinciden, verás "tirones": el personaje salta a la posición corregida.

6. Red y protocolo

6.1 El estado sincronizado (`RunState.ts`)

Colyseus sincroniza automáticamente un árbol de objetos definidos con `schema({...})`. Solo viaja lo que cambia. Los campos principales:

Esquema	Campos (resumen)
`RunState`	`seed`, `floor`, `tick`, `level`, `xp`, `floorTime`, `paused`, `started` y mapas: `players`, `enemies`, `projectiles`, `orbs`, `playerShots`, `hazards`, `chests`, `playerZones`.
`PlayerState`	posición y salto, `lastSeq`, vida, muerte, combo, arma (id, tier, naturaleza), multiplicadores (`moveSpeedMult`, `statusSpeedMult`, `cooldownMult` …), estados, habilidades (`AbilitySlotState`: id, rango, enfriamiento, cargas), equipo (`GearState` por ranura) y reliquia.
`EnemyState`	tipo, posición, vida, estado de IA y su tiempo, arma, tier de hechizo, élite, naturaleza, estados (quemando, ralentizado, congelado, aturdido).
otros	`ProjectileState`, `OrbState`, `HazardState` (zonas y rayos), `ChestState`, `PlayerShotState`, `PlayerZoneState`.

Los datos complejos (por ejemplo una pieza de equipo con sus stats) viajan como **texto JSON** en un campo `t.string()` (`GearState.data`) y el cliente los valida al parsear.

6.2 Mensajes (`MSG` en `protocol.ts`)

Mensaje	Dirección	Contenido
`input`	cliente → servidor	`PlayerInput`: `seq`, movimiento, `facing`, sprint, salto, ataque, E, máscara de habilidades, `viewTick`.
`welcome`	servidor → cliente	Tu clave y si la sala es sandbox.
`combat`	servidor → todos	`CombatEvent`: `damage`, `swing`, `levelUp`, `ability`, `chain`.
`floorWeapons`	servidor → un jugador	Sus objetos del suelo (armas, piezas, reliquias) con tiempo de vida.
`levelChoice` / `chooseCard`	ida y vuelta	Oferta de 3 cartas (con `offerId`) y la elección.
`dev`	cliente → servidor	Comando del panel F2 (solo en salas sandbox).

6.3 `PROTOCOL_VERSION`: cuándo subirlo

El cliente manda su versión al entrar; si no coincide con la del servidor, lo rechaza. **Súbelo en uno** cada vez que cambies el schema (agregar o quitar campos), el formato de un mensaje o el significado de un dato. Hay un test que fija el número en `packages/shared/test/cards.test.ts`: actualízalo también. Después de cambiarlo, recarga el navegador con Cmd+Shift+R.

6.4 Parseo defensivo en el cliente

`connection.ts` convierte el estado crudo en *snapshots* tipados (`PlayerSnapshot`, `EnemySnapshot`, `WorldSnapshot`…) y valida todo: números finitos, valores conocidos (una naturaleza desconocida pasa a `"none"`), JSON que no lanza errores. Si agregas un campo, agrégalo ahí con un valor por defecto seguro.

7. Determinismo y números aleatorios

La mazmorra se genera igual en cliente y servidor a partir de la semilla. Las tiradas del servidor (botín, IA, oleadas) usan generadores con semilla para que los tests sean reproducibles.

- **Nunca uses `Math.random()`** en `shared` ni en la simulación del servidor. Usa un `Rng` (`createRng(semilla)`).
- Cada sistema tiene su propio generador, creado en `RunRoom.onCreate` con la semilla de la sala combinada con una "sal" (por ejemplo `seed ^ ENEMY_RNG_SALT`). Así, agregar una tirada en un sistema no cambia las secuencias de los demás.
- **Cada tirada nueva cambia lo que sale después** en ese generador. Por eso, cuando agregas un sorteo, algunos tests con semilla pueden cambiar sus resultados esperados. Si quieres evitarlo, crea un generador nuevo con su propia sal, o sortea solo cuando haga falta (como la élite, que solo sortea desde el minuto 3).
- En el cliente sí puedes usar `Math.random()` para cosas puramente visuales (chispas, partículas), porque no afectan la partida.

8. Los sistemas del servidor

8.1 Combate (`CombatSystem.ts`)

- `CombatState` por jugador: clase, arma (id, tier, naturaleza), vida, combo, golpes pendientes recibidos, mods (de cartas, equipo y reliquia), habilidades, enfriamientos, bloqueo, bono del Grito, estados (quemadura, ralentización, aturdimiento) y efectos de equipo (`fx`).
- **Atacar**: `attack` empieza un golpe del combo del moveset del arma; el impacto se resuelve en `impactAt` de la animación, con arco, alcance y daño (`strikeDamage` × mods × bono).
- **Recibir daño**: los golpes enemigos se encolan con un tick de resolución (`queueEnemyHits`) y se aplican en `applyDue` : multiplicador de daño recibido, resistencia elemental, bloqueo frontal, estados del élite, Última resistencia, reflejo.
- **XP del grupo**: el nivel y la XP son **compartidos** (`grantXp`); al subir de nivel sube la vida máxima de todos.
- `setMods` recalcula la vida máxima con `maxHpFor` cuando cambian cartas, equipo o reliquia.

8.2 Habilidades (`AbilitySystem.ts`)

- Un jugador tiene hasta 4 habilidades activas en las teclas 1–4 (`abilitySlots`) más la pasiva (Onda de corte).
- `onInput` : detecta el flanco de la tecla, revisa enfriamiento/cargas, empieza la animación (`abilityAnim`) con sus tiempos de impacto (`impactTimes`) y resuelve los impactos vencidos (`resolveImpact` , un `switch` por habilidad).
- `step` (por tick): enfriamientos, recarga de cargas, Grito, ondas de corte, zonas del jugador (grietas).
- Los golpes a enemigos pasan por `enemies.strike` , que aplica la naturaleza del arma (quemar, ralentizar, encadenar).

8.3 Enemigos (`EnemySystem.ts`)

Cada enemigo tiene un estado de IA: `idle → chase → telegraph → recover` (y `dead`).

- **chase**: persigue al jugador vivo más cercano por recorrido, siguiendo el `FlowField` si no hay línea de vista. Los hechiceros se plantan a distancia preferida.
- **telegraph**: prepara el ataque (el cliente muestra el aviso). Al terminar golpea, dispara un orbe, marca una zona o dispara un rayo. `startTelegraph` es el único punto de entrada.
- **recover**: enfriamiento antes de volver a perseguir.
- Además: aturdimiento y congelamiento, empuje, separación entre enemigos, proyectiles, peligros (`hazards`), estados (quemadura, ralentización), élites (vida ×3, naturaleza, ataques alternados) y el sorteo de botín al morir (`rollDrop`).
- Los enemigos se crean con `createEnemy` desde `activateRooms` (salas), `spawnWave` (oleadas) o `spawnAt` (sandbox).

8.4 Oleadas, orbes y cartas

- `WaveDirector` : cada cierto intervalo (que se acorta con el reloj) crea un grupo a 8–30 casillas de recorrido del jugador, fuera de su vista, respetando el tope de enemigos vivos.

- **`OrbSystem`**: al morir, un enemigo deja orbes de XP; se atraen dentro del radio de imán y se recogen al tocarlos.
- **`LevelUpSystem`**: ofertas de 3 cartas (`rollOffer`), pausa del grupo, ofertas sin pausa para quien entra tarde o vuelve.

9. Dónde viven los datos del juego

La mayoría de los cambios de balance y contenido son **cambios de datos en `packages/shared`**. Estas son las tablas que más vas a tocar:

Quiero cambiar…	Archivo · símbolo
Vida del jugador por nivel	`combat.ts` · `CLASS_DEFS`
Vida, daño, velocidad, alcance, XP de enemigos	`combat.ts` · `ENEMY_DEFS`
Curva de XP por nivel	`combat.ts` · `xpToNext`
Armas (daño, alcance, velocidad) y combos	`weapons.ts` · `WEAPONS`, `MOVESETS`
Rarezas (multiplicador, color, probabilidad de soltar)	`weapons.ts` · `TIERS`
Qué arma lleva cada enemigo	`weapons.ts` · `ARMED_CHANCE`
Cartas de stat y su efecto	`cards.ts` · `STAT_CARDS`, `playerMods`
Habilidades (enfriamiento, daño por rango, clip)	`cards.ts` · `ABILITIES`
Ritmo de oleadas, escalado, tope de vivos	`waves.ts`
Hechizos por tier	`spells.ts` · `SPELLS`
Quemadura, ralentización, cadena; fuerza de élites	`natures.ts` · `NATURE`, `ELITE`
Equipo, legendarias, reliquias	`gear.ts` · `GEAR_BASE`, `LEGENDARY_POOL`, `RELICS`
Botín de cofres	`chests.ts` · `CHEST_LOOT`
Velocidades, salto, rango de recogida	`constants.ts`

Del lado del cliente, lo puramente visual también está en tablas: `render/looks.ts` (`ENEMY_LOOKS`: modelo, altura y animaciones por enemigo), `render/natureLooks.ts` (colores de naturalezas), `render/casterLooks.ts`, `render/tierGlow.ts`, `ui/abilityBar.ts` (`ABILITY_GLYPHS`) y `index.html` (colores y tamaños del HUD).

10. El cliente por dentro

10.1 El patrón de las vistas

Casi todo lo que se dibuja sigue la misma forma, por ejemplo `projectileViews.ts`, `hazardViews.ts`, `chestViews.ts`, `enemyViews.ts`:

```
class XViews {
  constructor(scene)                // crea materiales y texturas COMPARTIDOS
  push(world, receivedAtMs)         // llega un estado: crear/actualizar/marcar para borrar
por id
  render(renderAtMs)                // cada frame: posición interpolada, animaciones,
desvanecer
  setPaused(paused)                 // congelar partículas (updateSpeed = 0) en la pausa
  dispose()                         // liberar TODO: mallas, partículas, materiales, texturas
}
```

Para agregar algo nuevo al mundo, copia una vista existente parecida y conéctala en `main.ts` en los cuatro puntos: creación, `onWorld` (push), `frame` (render y setPaused) y el `return` de `startSession` (dispose).

10.2 Personajes y animaciones

- **`characterView.ts`**: instancia un modelo con esqueleto, lo escala a su altura, le cuelga el arma en la mano (`setWeapon`), reproduce clips, da acceso a los huesos (`boneMount`) y maneja estados como congelado.
- **`animLibrary.ts`**: carga los clips de `public/assets/mixamo/anims` y los reasigna a cualquier personaje con rig de Mixamo.
- **`animState.ts`**: decide qué clip toca (quieto, correr en 8 direcciones, saltar, golpe, habilidad, golpe recibido, muerte) y a qué velocidad, para que los pies no patinen.
- **`playerViews.ts`** / **`enemyViews.ts`**: arman cada personaje con sus extras (arma, barra de vida, auras, tintes, rótulos).

10.3 Efectos visuales

- `abilityFx.ts`: efectos de habilidades por línea de tiempo (`abilityTimeline`: anillos, ondas, grietas, destellos, estelas, chispas), con color según la naturaleza del arma.
- `projectileViews.ts`, `hazardViews.ts`, `chainFx.ts`: orbes y proyectiles por elemento, zonas, rayos, arcos eléctricos.
- `bodyAura.ts`: llamas sobre el cuerpo (élites, hechiceros, aura de conjunto del jugador).
- `gearTint.ts`: **shader** (plugin de material) que tiñe y da contorno a la zona del cuerpo de cada pieza de equipo.
- `natureLooks.ts`, `tierGlow.ts`, `casterLooks.ts`: colores y brillos de armas, rarezas y hechiceros.
- `fxTextures.ts`: texturas generadas en código (chispa, llama, gradientes), compartidas.

10.4 Reglas del render

> **Sin luces nuevas.** La escena tiene un presupuesto de luces fijo (`render/lighting.ts`, antorchas). Todo efecto se hace con materiales **emisivos** (que brillan solos), transparencias y partículas.

- **Materiales compartidos**: crea un material por tipo/color/tier y reúsalo; nunca uno por instancia (cuesta memoria y llamadas de dibujo).
- **Reloj de juego**: anima con el tiempo que te pasan a `render`, no con `performance.now()`, para que todo se congele en la pausa.
- **Liberar todo**: cada malla, partícula, material y textura creada debe destruirse en `dispose`. Los tests con `NullEngine` cuentan que no queden sobrantes.
- **Nada de asignar memoria por frame** si se puede evitar (vectores, closures, strings en bucles calientes).

10.5 La interfaz (DOM)

El HUD no es 3D: son elementos HTML encima del canvas, con estilos en `index.html`. `ui/hud.ts` (retrato, vida, XP, arma, estados, iconos de equipo y tooltips), `ui/abilityBar.ts`, `ui/cardPanel.ts`, `ui/compass.ts`, `ui/minimap.ts`, `ui/devPanel.ts`. Los iconos de equipo son SVG en `ui/gearIcons.ts`. Se actualiza el DOM solo cuando el contenido cambia.

10.6 Modelos y animaciones nuevos

1. Descarga el personaje o la animación de Mixamo como .glb (con esqueleto de Mixamo).
2. Ponlo en la carpeta de trabajo (por defecto `.superpowers/kingslayer-work/glb/`) y agrégalo a las listas de `scripts/assets/build-mixamo.mjs`.
3. Corre `node scripts/assets/build-mixamo.mjs`: regenera `public/assets/mixamo/` y su `manifest.json`.
4. Usa el id del clip en un moveset (`weapons.ts`), una habilidad (`cards.ts`, campo `clip`) o un look de enemigo (`render/looks.ts`).
5. Para ajustar el agarre de un arma, usa la página `packages/client/dev/grip.html` (abre `localhost:5173/dev/grip.html`).

11. Recetas paso a paso

Cada receta indica los archivos en orden. Al terminar: `npm test`, tipos y probar en `?sandbox=1`.

11.1 Cambiar el balance

Edita el número en su tabla (capítulo 9). Si un test fija ese número, actualízalo. No hace falta tocar el protocolo.

11.2 Agregar una carta de stat

1. `shared/src/cards.ts`: agrega el id a `StatCardId` y su entrada en `STAT_CARDS` (nombre y texto).
2. Si es un efecto nuevo, agrega un campo a `PlayerMods` y calcúlalo en `playerMods` (y en `combinedMods` de `gear.ts`, que suma cartas, equipo y reliquia).
3. Úsalo en el servidor donde corresponda (por ejemplo en `CombatSystem`).
4. Si el cliente necesita saberlo para predecir (como la velocidad), agrégalo a `PlayerState`, sincronízalo en `syncPlayer`, léelo en `connection.ts` y sube `PROTOCOL_VERSION`.
5. Tests en `shared/test/cards.test.ts`.

11.3 Agregar una habilidad

1. `shared/src/cards.ts`: agrega el id a `AbilityId` y su `AbilityDef` en `ABILITIES` (tipo, enfriamiento, duración de animación, clip, rangos, texto).
2. `server/src/sim/AbilitySystem.ts`: sus tiempos de impacto en `impactTimes` y su efecto en `resolveImpact` (daño con `enemies.strike`, aturdir, empujar…).
3. El clip de animación (ver 10.6) y, si hace falta, la lógica de animación en `render/animState.ts`.
4. `client/src/ui/abilityBar.ts`: su símbolo en `ABILITY_GLYPHS`.
5. `client/src/render/abilityFx.ts`: su efecto visual en `abilityTimeline`.
6. `LocalAbility` predice cualquier habilidad activa sin cambios, salvo que tenga mecánicas nuevas (cargas, costos).
7. Tests en `server/test/AbilitySystem.test.ts`. Pruébala con F2 → Cartas → "Dar carta".

11.4 Agregar un tipo de enemigo

1. `shared/src/combat.ts`: el tipo en `EnemyKind` y sus números en `ENEMY_DEFS`.
2. `shared/src/weapons.ts`: su entrada en `ARMED_CHANCE` (`null` si no lleva arma).
3. Si es hechicero: `spells.ts` e `isCaster`; su ataque en `EnemySystem.startTelegraph` y el final de la telegrafía.
4. Dónde aparece: `waves.ts` (`waveKinds` / `casterKinds`) o `EnemySystem.activateRooms`.
5. Cliente: su look en `render/looks.ts` (`ENEMY_LOOKS`: modelo, altura, clips) y el modelo en `render/assets.ts` si es nuevo.
6. Panel F2: su etiqueta en `ENEMY_LABELS` (`ui/devPanel.ts`).
7. Sube `PROTOCOL_VERSION` (el cliente debe conocer el tipo nuevo).

11.5 Agregar un arma

1. El modelo .glb en `build-mixamo.mjs` (o en `public/assets`) y su id en `render/assets.ts`.
2. `shared/src/weapons.ts`: el id en `WeaponId` y su entrada en `WEAPONS` (nombre, género gramatical, moveset, daño, multiplicadores).
3. Si usa un estilo nuevo, un `MOVESETS` nuevo con sus clips y su combo.
4. Ajusta el agarre en `dev/grip.html` y guárdalo en la tabla de looks del arma.
5. Sube `PROTOCOL_VERSION`.

11.6 Agregar una pieza de equipo, una legendaria o una reliquia

- **Reliquia**: `gear.ts` → `RelicId` y `RELICS`; su efecto en `combinedMods` (si es un modificador) o en el servidor (si es una regla, como el Colmillo del vampiro).
- **Efecto legendario**: `gear.ts` → `LegendaryId`, `LEGENDARY_POOL`, `LEGENDARY_TEXT`, `LEGENDARY_ABILITY`; su efecto en el sistema que corresponda (habilidades o combate), preguntando si el jugador lo tiene (`legendaries`).
- **Ranura nueva** (como se hizo con el cinturón): `GearSlot` y `GEAR_SLOTS`, stat principal en `GEAR_MAIN`, icono en `ui/gearIcons.ts`, zona del cuerpo en `render/gearTint.ts`, objeto del suelo en `render/floorItemViews.ts`, etiqueta del panel. El resto (botín, equipar, sincronizar) es genérico.

11.7 Agregar un comando al panel F2

1. `shared/src/sandbox.ts`: el caso en `DevCommand` y su validación en `parseDevCommand` (rechaza todo lo que no sea válido).
2. `server/src/rooms/RunRoom.ts`: el `case` en `applyDev`.
3. `client/src/ui/devPanel.ts`: el botón o interruptor (`this.button`, `this.toggle`, `this.select`).
4. Test en `shared/test/sandbox.test.ts` y, si aplica, en la sección "sandbox" de `RunRoom.test.ts`.

11.8 Sincronizar un dato nuevo del servidor al cliente

1. Agrega el campo al schema en `server/src/rooms/RunState.ts`.
2. Asígnalo cada tick en `syncState.ts` (`syncPlayer` o `syncWorld`).
3. Léelo en `client/src/net/connection.ts` (en el snapshot, con valor por defecto seguro).
4. Úsalo en la vista o el HUD.
5. Sube `PROTOCOL_VERSION` y su test.

11.9 Agregar un efecto visual

1. Si es de una habilidad: un paso más en `abilityTimeline` (`abilityFx.ts`).
2. Si es algo del mundo que viene del servidor: una vista nueva con el patrón de 10.1, conectada en `main.ts`.
3. Si es un evento puntual: un `CombatEvent` nuevo (servidor lo emite, `isValidCombatEvent` lo valida, `conn.onCombat` lo dibuja).
4. Respeta las reglas de 10.4 y agrega un test con `NullEngine` que cree y libere sin sobrantes.

12. Tests

Hay unos 800 tests con **Vitest**, uno o varios archivos por módulo, en `packages/*/test/`.

- **shared**: funciones puras (generador de mazmorras, movimiento, fórmulas, sorteos con semilla).
- **server**: los sistemas por separado (`EnemySystem.test.ts`, `CombatSystem.test.ts`…) y la sala completa (`RunRoom.test.ts`), que levanta un servidor de prueba y conecta clientes reales.
- **client**: lógica pura (predicción, animaciones, formatos del HUD) y vistas con `NullEngine` de Babylon (sin navegador) para comprobar que se crean y liberan bien.

12.1 Ayudas de `RunRoom.test.ts`

```
const ana = await join("Ana", "t-ana");   // conecta un cliente de prueba
await ana.welcome;
teleport("ana", { x, z });                 // mueve al jugador en el servidor
record("ana").combat.hp = 10;              // acceso directo al registro
await ticksUntil(() => room.state.paused, 30);  // espera ticks hasta que se cumpla algo
// sección "sandbox": sandboxRoom(), dev(client, cmd), settle(n)
// startRun(): marca la partida como empezada (sin sellos)
```

> **Un buen hábito:** escribe primero el test que falla, después el código. Si cambias una regla compartida, prueba ambos lados (servidor y predicción del cliente).

13. Reglas y trampas comunes

Revisa esta lista antes de cada cambio:

#	Regla	Por qué
1	Sin `Math.random()` en `shared` ni en la simulación del servidor.	Determinismo: mazmorra igual en ambos lados y tests reproducibles.
2	El servidor decide; el cliente solo predice y dibuja.	Evita trampas y estados distintos entre jugadores.
3	Lo que se predice se calcula igual en ambos lados (mejor: una función de `shared`).	Si no, hay tirones al reconciliar.
4	Cambias el schema o un mensaje → sube `PROTOCOL_VERSION` y su test.	Un cliente viejo no debe entrar a un servidor nuevo.
5	Todo lo que depende del tiempo respeta la pausa.	En la elección de cartas el juego debe quedar quieto.
6	Sin luces nuevas; emisivos y partículas.	Presupuesto de rendimiento.
7	Materiales compartidos y `dispose` de todo.	Memoria y fugas en partidas largas.
8	Valida en el servidor todo lo que manda el cliente.	Nunca confíes en el navegador.
9	El orden del input importa: mover → combo → habilidades → recoger → atacar.	El cliente predice en el mismo orden.
10	Una tirada nueva en un generador cambia las siguientes.	Puede romper tests con semilla; usa otra sal si quieres aislarla.

13.1 Problemas que ya aparecieron

- "**No se pudieron cargar los modelos**" al guardar varios archivos a la vez: Vite recargó con archivos a medio escribir. Recarga con Cmd+Shift+R.
- **El juego no entra después de un cambio**: probablemente cambió `PROTOCOL_VERSION`; recarga y verifica que el servidor se reinició.
- **Un efecto no se congela en la pausa**: usa el reloj que llega a `render` y conecta `setPaused`.
- **Partículas que dejan de emitir**: en Babylon, tras una ráfaga manual (`manualEmitCount`), hay que volver a ponerlo en `-1`.
- **Clonar un material clona sus texturas dinámicas en blanco**: comparte el material y usa `mesh.visibility` para desvanecer.

14. Flujo de trabajo y documentos

- `docs/specs/`: una especificación por fase, con la intención, las decisiones y lo que queda fuera. Léela antes de tocar una mecánica para entender el porqué.
- `docs/plans/`: el plan de cada fase dividido en tareas, con los archivos, interfaces y tests de cada una. Sirven de ejemplo de cómo se agregó cada cosa.
- **Git**: trabaja en una rama (`git switch -c mi-cambio`), commits chicos con mensaje claro, `npm test` antes de mezclar a `main`.
- **Pruebas manuales**: `?sandbox=1` + panel F2 para llegar rápido a cualquier situación; dos pestañas con nombres distintos para probar co-op; `?lag=150` para probar la predicción.

14.1 Un ejemplo completo: subir el daño del Torbellino

1. Abre `packages/shared/src/cards.ts`, busca `whirlwind` y cambia `damageMult` de sus rangos.
2. Corre `npm test -w @roguelike/shared`; si un test fija el valor, actualízalo.
3. Con `npm run dev` corriendo, el servidor se reinicia solo. Abre `localhost:5173/?sandbox=1`, F2 → Cartas → "Torbellino III" → Dar carta → Enemigos → crear 5 brutos → tecla 1.

15. Glosario

Término	Significado
Tick	Un paso de la simulación del servidor (1/30 s).
Schema	Estructura de datos de Colyseus que se sincroniza sola a los clientes.
Parche	Los cambios del schema que el servidor manda cada 50 ms.
Predicción	El cliente simula tu personaje sin esperar al servidor.
Reconciliación	Corregir la predicción partiendo del estado real y reaplicando los inputs pendientes.
Interpolación	Dibujar a otros entre dos estados recibidos, 100 ms en el pasado.
`seq` / `lastSeq`	Número de cada input / último input que el servidor procesó.
`viewTick`	El tick que veías al hacer clic; sirve para rebobinar al golpear.
Telegrafía	El aviso de un ataque enemigo antes de que golpee.
Peligro (hazard)	Zona del piromante o rayo del invocador: aviso y luego daño.
Naturaleza	Elemento (fuego, hielo, rayo) de un arma o un élite.
Tier	Rareza: 1 oxidado, 2 templado, 3 rúnico, 4 legendario.
Sal (salt)	Número que se combina con la semilla para que cada sistema tenga su propio generador.
NullEngine	Motor de Babylon sin pantalla, usado en los tests del cliente.
Sandbox	Sala de pruebas con el panel F2.

Esta guía describe el estado del código a la fecha. Si algo no coincide, la referencia es el código y los documentos de `docs/specs` y `docs/plans`.